VIEWER PRODUCT

Integration Guide

For an engineer at a common data environment — Asite, Procore, Dalux, or your own — who has to make this viewer open your customers' documents.

> **Read section 1 before planning anything.** One of the four exchanges an integration needs is built. The other three are not, and this guide says so rather than describing an interface you would then fail to find.

Companion document: **Technical Architecture**, for how the thing works internally.

Contents

1. What you can build against today

HOST CDE	EXCHANGE	VIEWER PRODUCT
Their document store SharePoint / S3 / Blob	1. mount the viewer ✗ NOT BUILT — no embed surface	viewer-core 6 components · 6 services
Their identity provider Entra / Okta / own	2. identify the user ✗ NOT BUILT — no token exchange	Conversion pipeline fetch → scan → render
Their web interface where the viewer appears	**3. hand over the document →** **✓ BUILT** **POST /api/conversions**	Markup store tied to our user table
	← 4. return the markups ✗ NOT BUILT — 17 operations undecided	

Figure 1 — Three of the four exchanges do not exist. The viewer can already take a document from anyone's storage; it cannot yet be mounted, be told who is looking, or give anything back.

So: you can convert and render your customers' documents through our API today, inside your own interface, using your own rendering. Embedding our viewer is specified but not yet built — the contract is settled (section 5), the code is not.

If that is enough — and for a "preview any file format" feature it often is — section 3 is a complete, working integration. If you need markup round-tripping, section 6 is what you are waiting for.

2. The shape of it

Your CDE keeps its documents and its users. The viewer borrows a document briefly, converts it, and discards it.

We never hold your credentials and never learn which storage platform your link points at. SharePoint, S3, Azure Blob and GCS all reduce to "a URL that works for fifteen minutes", which is the property that makes this portable across CDEs rather than a per-vendor connector.

3. Converting a document from your storage

#	STAGE	WHAT HAPPENS
1	**Host mints a link**	Short-lived, single-object, your credentials. We never hold them.
2	**Submit**	POST /api/conversions { sourceUrl, targetFormat } — 202 + Location header, under one second
3	**Destination policy**	Resolves the hostname and validates the ADDRESS, not the string. Refuses loopback, link-local, 169.254.169.254, RFC 1918, .internal. Redirects are NOT followed.
4	**Bounded fetch**	Size cap enforced during the stream, not after. Connect and read timeouts.

#	STAGE	WHAT HAPPENS
5	**Quarantine + scan**	Magic-byte type check, then ClamAV. Not referenced by the application until clean.
6	**Convert**	Sandboxed, resource-limited, network-isolated worker, out of process.
7	**Poll**	GET /api/conversions/{jobId} — PENDING, RUNNING, then SUCCEEDED / FAILED / CANCELLED (terminal)
8	**Collect**	GET /api/conversions/{jobId}/content — streamed

Figure 2 — The one exchange that is built, end to end. Stages 1, 7 and 8 are yours; 2 to 6 are ours. Always send an Idempotency-Key so a timed-out retry returns the same job rather than converting twice.

3.1 Mint the link

Generate a short-lived, single-object download URL with your own credentials — a Microsoft Graph download URL, an S3 presigned GET, an Azure blob SAS, a GCS signed URL. Keep the expiry tight; we fetch once, immediately.

Do **not** send us a permanent URL, a URL that needs a credential we would have to store, or a path on your internal network — section 3.5 explains what happens.

3.2 Submit

```
curl -X POST https://viewer.example.com/api/conversions \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 8f14e45f-ceea-467a-9f3a-1d2c9b7e4a51" \
  -d '{
        "sourceUrl": "https://files.example.test/drawings/site-plan.dwg?token=synthetic",
        "targetFormat": "PDF"
      }'

202 Accepted
{ "jobId": "3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40", "status": "PENDING",
  "sourceHost": "files.example.test", "targetFormat": "PDF" }
```

Both fields are required. `targetFormat` accepts `PDF` and nothing else today — it is in the contract so that adding a second output is additive rather than breaking. The `jobId` is a UUID; use it verbatim on every other endpoint.

Requires the `document:convert` permission. Returns in under a second — the work has not happened yet, and that is the design: an endpoint either answers in under a second or hands back a job id in under a second.

> **Always send `Idempotency-Key`.** A timed-out retry then returns the same job rather than converting the same file twice, and conversion is the expensive operation in this system.

Note what is not echoed back: the URL. Only its host is kept on the job record. The link is a bearer credential, so it is never written to the database, a log, or a message — which also means you cannot read it back to find out what you sent.

3.3 Poll

```
curl https://viewer.example.com/api/conversions/3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40 \
  -H "Authorization: Bearer $TOKEN"

200 OK
{ "jobId": "3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40", "status": "SUCCEEDED",
```

```
  "sourceHost": "files.example.test", "sourceFileName": "site-plan.dwg",
  "targetFormat": "PDF", "resultSizeBytes": 486213, "failureReason": null }
```

`PENDING` while queued, `RUNNING` while working, then one of `SUCCEEDED`, `FAILED` or `CANCELLED`. **Those three are terminal and never change again**, so stop polling when you see one. On `FAILED`, read `failureReason`.

There is no progress percentage and no sub-stage — the job reports the state it is in, not how far through it is. Back off as you poll; a large IFC will not finish in the first second, and hammering the endpoint only costs you rate limit.

3.4 Collect

```
curl https://viewer.example.com/api/conversions/3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40/content \
  -H "Authorization: Bearer $TOKEN" -o converted.pdf
```

Streamed, with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff`. Cancel an in-flight job with `DELETE /api/conversions/{jobId}`.

3.5 What we refuse, and why

Your URL is dereferenced by our server, which makes it an SSRF vector. Every submission goes through a destination policy that:

- **resolves the hostname and validates the resolved address**, not the string — a name that resolves to `169.254.169.254` is refused however it is spelled;
- refuses loopback, link-local, RFC 1918 (`10/8`, `172.16/12`, `192.168/16`), `::1`, and `.internal` names;
- **does not follow redirects at all.** Not “follows a capped number and re-validates” — none. Your link must resolve to the bytes directly;
- requires `https` unless the deployment has explicitly opted out;
- enforces the deployment’s list of permitted storage hosts, where one is configured;
- bounds the fetch: size enforced **during** the stream, plus connect and read timeouts.

> **The redirect rule is the one most likely to break you.** Plenty of storage 302s to a CDN. Test the exact URL you intend to send, not a URL that eventually reaches the bytes. And ask which hosts are allow-listed on the deployment before you integrate.

After the fetch: content type is decided by **magic bytes**, never by your `Content-Type` header or the file extension; the file lands in a quarantine prefix; ClamAV scans it; only then does it convert. An infected file is deleted and the event audited.

3.6 Errors

RFC 9457 problem documents throughout, with a `traceId` you can quote.

```
{ "type": "/problems/fetch-not-permitted",
  "title": "Fetch not permitted",
  "status": 422,
  "detail": "The source URL resolves to a private address (10.0.4.19).",
  "traceId": "4f8a1c2e9b7d6a5f3e2d1c0b9a8f7e6d" }
```

Endpoint	Status	Means
POST /api/conversions	202	Accepted. The Location header names the job to poll — use it rather than assembling the URL yourself
	422	The link may not be fetched: wrong scheme, an unlisted host, or a refused destination. This is the SSRF refusal —

Endpoint	Status	Means
		it is a 422, not a 403
	429	The conversion queue is full, not a per-caller rate limit. Retry after the interval in Retry-After
GET /api/conversions/{jobId}	**404**	No such job is visible to you. Note the wording: another tenant's job is invisible, not forbidden
DELETE /api/conversions/{jobId}	**404**	As above
GET /{jobId}/content	**409**	The job has not succeeded, so there is nothing to download yet. Poll to a terminal state first

Table 1 — Status codes you will actually meet.

Read `detail` and show it. It is written to be shown. Quote `traceId` to support — it is the only thing that ties your failure to our logs.

4. Formats

Input	Status	Route	Notes
PDF	**Supported**	Passed through; pdfjs-dist	OCR available for scanned pages, adds an invisible text layer
DXF	**Supported**	ezdxf → SVG or PDF	Text is real and searchable in both
Office	**Supported**	LibreOffice → PDF	docx, xlsx, pptx
IFC	**Supported**	Geometry + hierarchy tree	Tree is the primary interface, not a fallback
DWG	**Hosted only**	→ DXF, then as above	Works in our hosted service. NOT shippable on-premises — see below
DWF	**Not supported**	—	No route exists. Autodesk's Design Web Format is not read by any component of the pipeline
RVT / RFA	**Not supported**	Detected and refused	Recognised by OLE2 magic bytes and rejected with REVIT_BINARY. Identified only so the error is clean

Table 2 — What the pipeline actually accepts. Four supported, one hosted-only, two not supported at all.

> **DWG has no clean licensing path for a distributed product.** The two converters have opposite problems: LibreDWG is GPL-3.0 and ships in our image (so distribution owes corresponding source we do not currently provide), and the ODA File Converter cannot be redistributed at all. In our hosted service DWG works. For an on-premises install, **assume DWG is unavailable unless your contract says otherwise** and you have obtained an ODA licence yourself. Tracked as ADR 13, referred to counsel, unresolved. Do not plan a DWG workflow on the assumption this resolves in your favour.

4.1 Why RVT and DWF are absent, and what it would take

These two are named separately because they are the ones most often assumed, and because the reason they are missing is not “nobody got to it yet”.

RVT (and RFA) is a proprietary Autodesk binary with no open reader. The pipeline detects it by OLE2 magic bytes and refuses it deliberately, so you get a clean `REVIT_BINARY` error rather than a corrupt render. Every route to supporting it is commercial.

Route	What it costs
Autodesk Platform Services (formerly Forge)	A cloud API. It SENDS THE MODEL TO AUTODESK, which collides directly with the data-residency and sovereignty position — for a Defence or IRAP-scoped tenant it is not merely a cost question, it is prohibited
Revit itself, plus an export plugin	Windows hosts, a licence per seat, and a rendering farm that is not the product’s architecture
A commercial SDK (ODA BimRv or equivalent)	A vendor relationship and a per-deployment fee, with the same redistribution question ADR 13 is already stuck on

Table 3 — The three routes to RVT, and why none is free.

> The sovereignty collision is the important one. **The cheapest route is the one we can least use**, because the tenants most likely to hold Revit models are also the ones whose data must not leave the jurisdiction.

DWF is unimplemented rather than refused. It is a ZIP container holding W2D and W3D streams, and reading it is ordinary engineering rather than a licence problem — but no component of the pipeline knows the format today, so a DWF submission will fail as an unrecognised type. If DWF matters to you, say so: it is schedulable work in a way RVT is not.

If you need either, raise it as a commercial requirement, not a bug. The answer involves a vendor contract and, for RVT, a residency decision that engineering cannot take alone.

5. Embedding — decided, not yet built

The contract is settled and specified. An iframe and a versioned postMessage protocol; the viewer authenticates nobody and authorises nothing. You mint a short-lived URL for the document, tell the viewer a name to display, and receive what the user did — every operation that changes anything comes back to you to authorise server-side.

Recorded as ADR 14 (`cde-platform`, `docs/adr/0014-viewer-embed-and-identity-contracts.md`), with the message-level detail in `docs/viewer-embed-protocol.md`. That document is written to be built against and is stable enough to design around, though the code does not exist yet.

Three constraints remain true of the code as it stands today.

Constraint	Reality today
Framing	Content-Security-Policy: frame-ancestors 'none'. A cross-origin iframe will not render. Becomes a per-tenant allow-list on the embed route only

Constraint	Reality today
Cross-origin XHR	The CORS source registers no origin at all unless a deployment names one in full
Session	A bearer token from our own /api/auth/login, against our own user table. No cookie mode, no silent SSO, no token exchange

Table 4 — What is still true of the code, now that the contract is decided.

The only path that works today is **serving our Angular build from your own origin, behind the same web tier that proxies `/api`**. Same-origin needs no CORS entry, no framing relaxation, and no cross-origin token handling. It is also a deployment of our application into your infrastructure, which is probably not what you meant by “integrate”.

> **If you only need viewing**, the honest recommendation is still to skip the embed: use section 3 to convert, and render the resulting PDF or SVG in your own interface with your own viewer. That avoids all three constraints and is a complete feature today rather than a contract to wait for.

5.1 What the identity gap actually means

Not “you need to configure SSO”. There is nothing to configure. In the code as it stands, a markup’s author is a foreign key into our user table, so **your user must exist as a row in our database before they can annotate anything** — which means shadow-provisioning your users into our system, a data-protection conversation before it is an engineering one.

TODAY	NEEDED
Priya, at Asite host identity	Priya, at Asite signed assertion
↓ **blocked**	↓
Annotation.author @ManyToOne User	Annotation.authorRef opaque value
↓	↓
OUR users table she must exist as a row	Issuer + subject no row required
Integration means shadow-provisioning every host user into our database.	The host stays the system of record for who its people are.

Figure 3 — The difference is one field. Annotation.author is a foreign key into our own user table, so a markup cannot exist without a local user row.

That is what the contract removes. The viewer is told a display name and an opaque subject id, both untrusted and used only for presentation, and the author is stamped by whoever persists the markup — you. You stay the system of record for who your people are, and nothing about your users reaches our database.

> You do **not** need to mint a JWT, and you do not need an identity provider we can talk to. If an earlier version of this guide told you to prepare a signed assertion with a stable subject claim, disregard it — that was written while the question was open, and the answer went the other way.

6. What to plan for, not against

The contract has landed. It is specified, not implemented — do not build against it yet — but it is settled enough to design around, and these are no longer guesses.

- **Embedding is an iframe** with a per-tenant `frame-ancestors` allow-list. Not a web component: the viewer renders untrusted documents, and sharing your origin would mean a document that escapes the PDF renderer runs with your session on your domain.
- **Identity is three untrusted fields** — a display name, an opaque subject id, and capability flags that decide which controls render and nothing else. No signed assertion, no viewer session, no JWT. Sending no identity at all is a supported read-only deployment.
- **Markup comes back as events** on the message channel, with the geometry as an opaque string you store and hand back. There is no author field — you stamp that.
- **Document identity is yours.** Pass an `externalId` in the handshake and it comes back on every event, so you never hold a map between your id and ours.

Full message shapes: `docs/viewer-embed-protocol.md`.

7. Obligations you inherit

Accessibility. WCAG 2.2 AA is a procurement gate in the UK, EU, Australia and the US, and an embedded viewer becomes part of *your* conformance claim. If you embed our WebGL model view, you inherit the requirement for an equivalent accessible route to the same information — we provide the hierarchy tree for exactly this reason, and it needs to remain reachable in your integration. Exports must be tagged and PDF/UA-conformant.

Trademarks. “Works with Microsoft SharePoint” is nominative fair use. “Microsoft-approved”, their logo, or any implication of partnership is not. The same applies to Amazon, Google and Autodesk. Integration documentation is where this goes wrong most easily, in both directions.

> **Certification claims.** Do not describe this product as SOC 2, ISO 27001 or IRAP certified. Those certifications are not currently held, and stating otherwise is a misrepresentation with regulatory consequences rather than a marketing stretch.

8. Integration checklist

Today

- Can you mint a short-lived, single-object download URL from your storage?
- Does that URL resolve to the bytes directly, with no redirect?
- Do you have somewhere to hold a job id between submit and collect?
- Do you generate an `Idempotency-Key` per logical submission?
- Do you back off while polling, and honour `Retry-After` on `429`?
- Do you surface `detail` and `traceId` from problem documents to your support path?
- Have you confirmed your DWG position?

Before you scope an embedded integration

- Have you read the embed protocol? (`docs/viewer-embed-protocol.md`)

- Can you mint a short-lived URL for a document, and serve a page that frames us?
- Do you have somewhere to store markup that is not the rendered file?
- Do you know which of the 17 document operations you need to implement as callbacks?

> The embed is specified but not built, so the integration you can ship **today** is still section 3 and nothing else. That is a real feature and it works — it is just smaller than “embed the viewer”, and worth scoping as what it is.